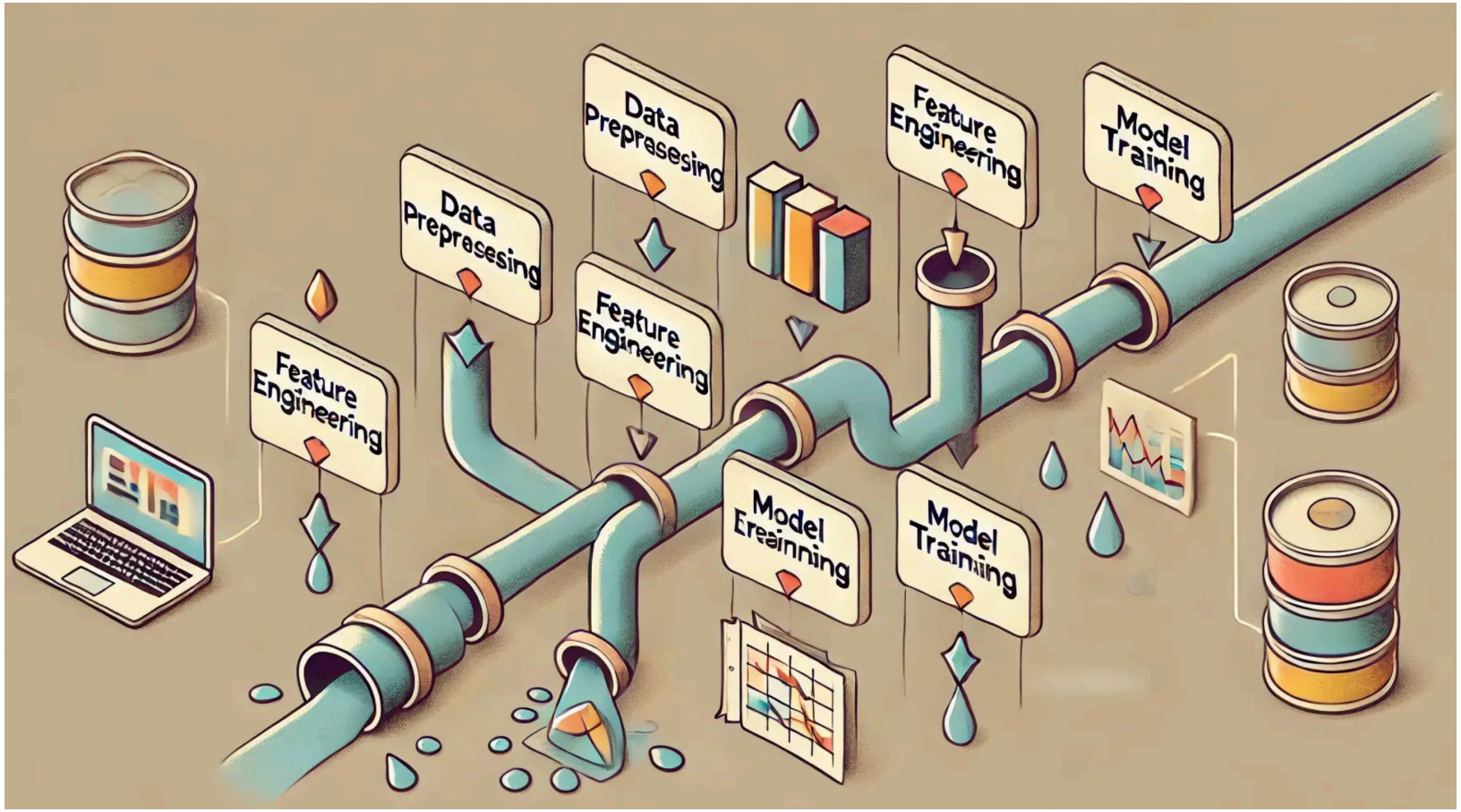
Data Leakage



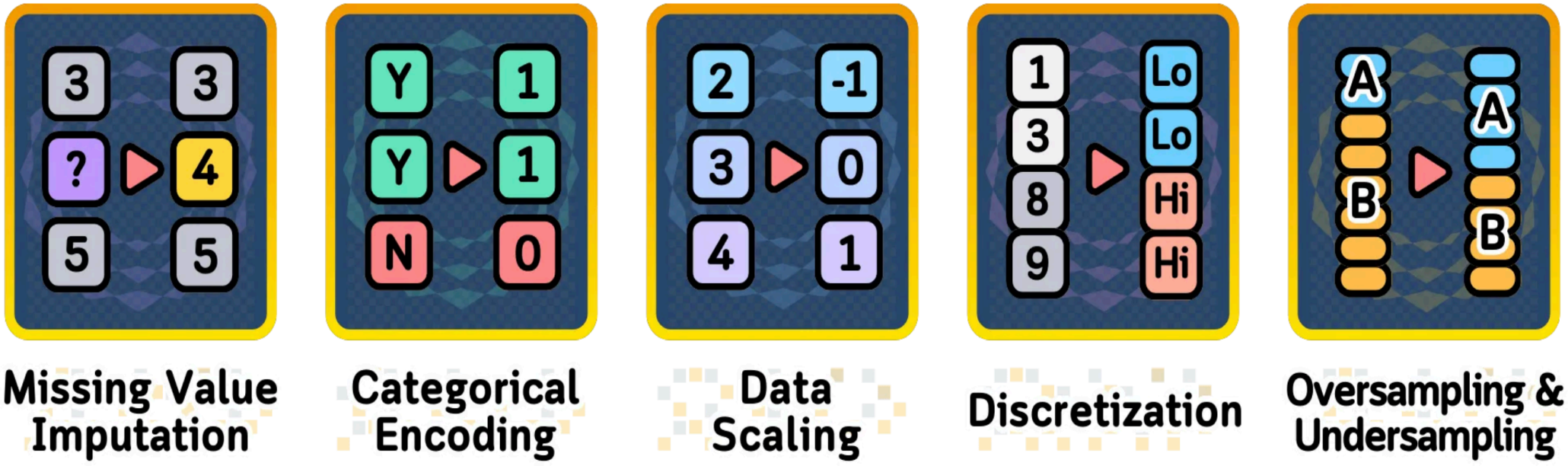
www.towardsdatascience.com

Què és?

La filtració de dades es produeix quan la informació de les dades de prova (*test set*) es filtra a les dades d'entrenament (*train set*) durant els passos de preparació de les dades.

Quan això succeeix, el model aprèn de les dades de prova que no havia de veure, fent que els resultats de prova siguin massa bons/inflats.

Quàn?


3
3
?
4
5
5
Missing Value Imputation
Y
1
Y
1
N
0
Categorical Encoding
2
-1
3
0
4
1
Data Scaling
1
3
8
9
Lo
Lo
Hi
Hi
Discretization
A
B
A
B
Oversampling & Undersampling

Missing value imputation

```
SimpleImputer(strategy='mean')
```

```
KNNImputer()
```

```
SimpleImputer(strategy='ffill')
```

```
SimpleImputer(strategy='bfill')
```

```
SimpleImputer(strategy='constant', fill_value=value)
```

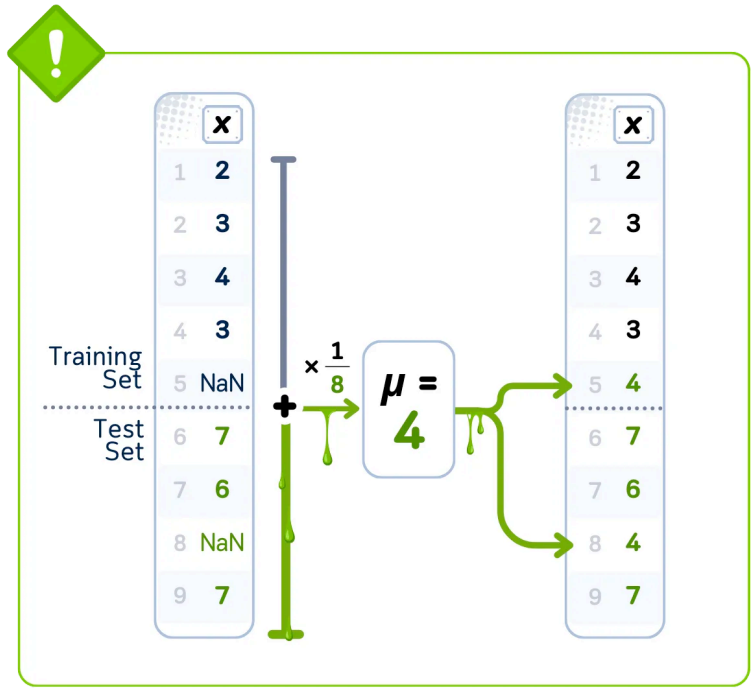


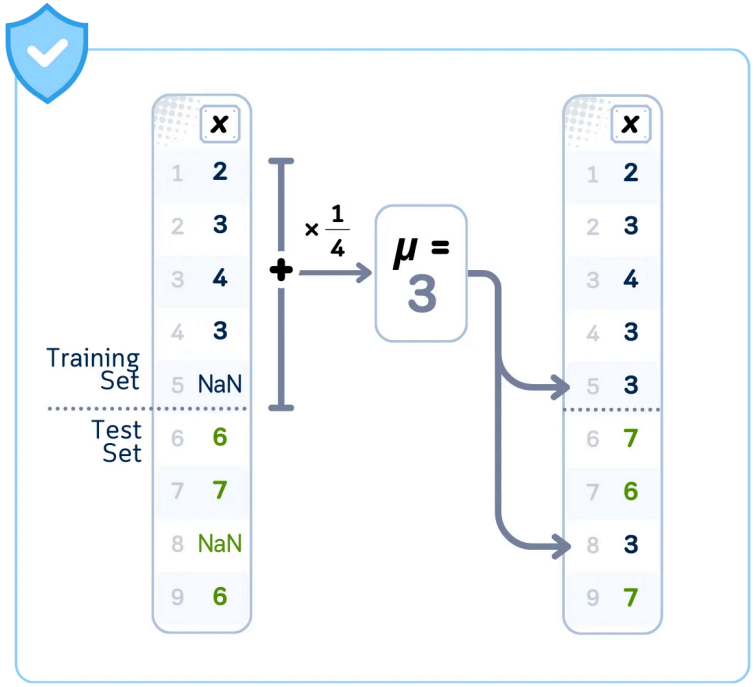
`SimpleImputer(strategy='mean')`

🚨 El PROBLEMA: calcular valors mitjans mitjançant el dataset al complet

❌ Què estem fent malament? Calcular els valors d'ompliment utilitzant tant les estadístiques de ***training*** com de ***test***

💥 La conseqüència: les dades de ***training*** contenen valors mitjans influenciats per les dades de ***test***



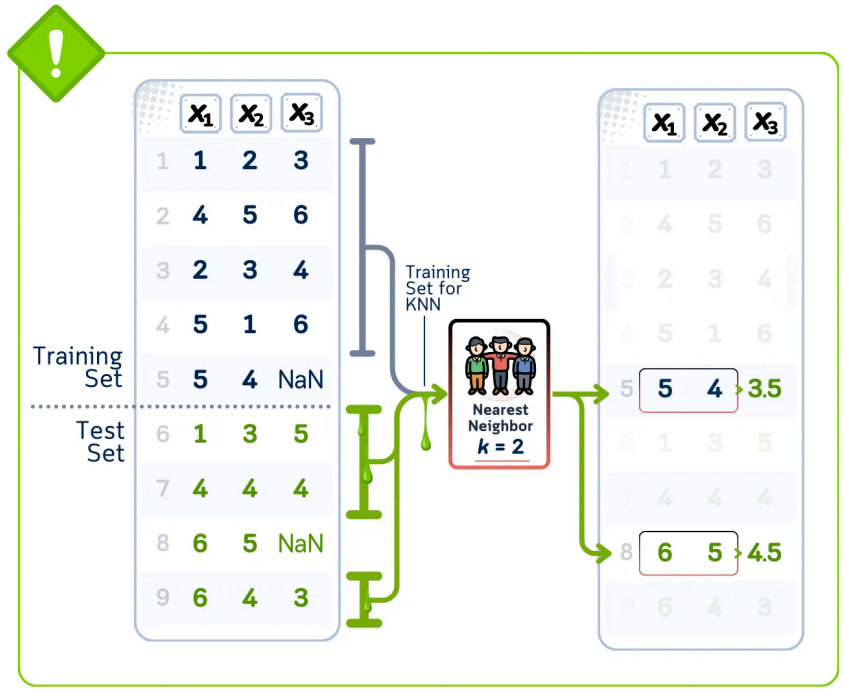


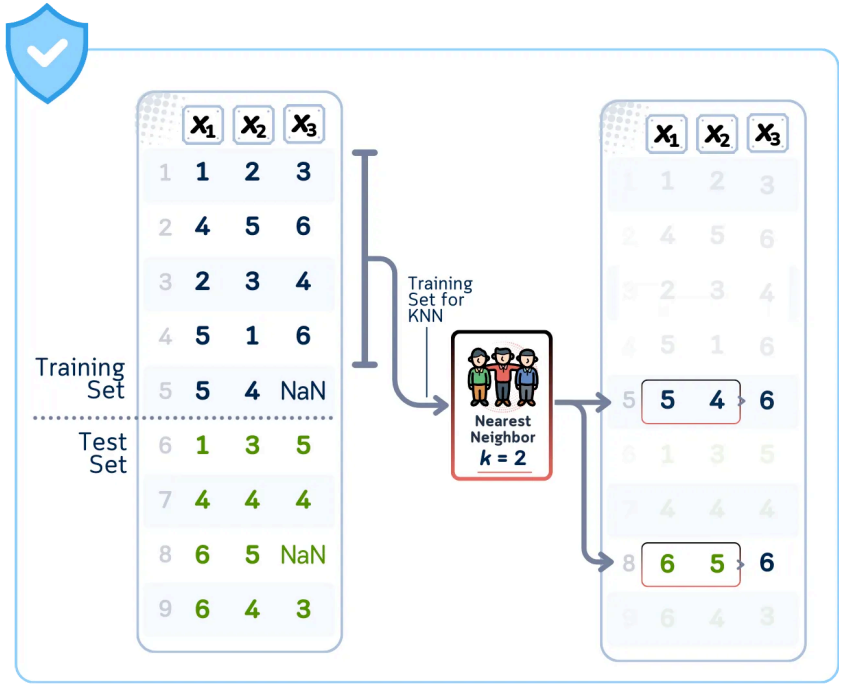
`KNNImputer()`

🚨 EL PROBLEMA: trobar veïns mitjançant el dataset al complet

❌ Què estem fent malament? Usar mostres de conjunt de ***test*** com a possibles veïns per a la imputació

💥 La conseqüència: els valors que falten s'han completat mitjançant la informació del conjunt de ***test***





Categorical Encoding

`TargetEncoder()`

`OneHotEncoder()`

`OrdinalEncoder()`

`LabelEncoder()`

`OrdinalEncoder(categories=[ordered_list])`

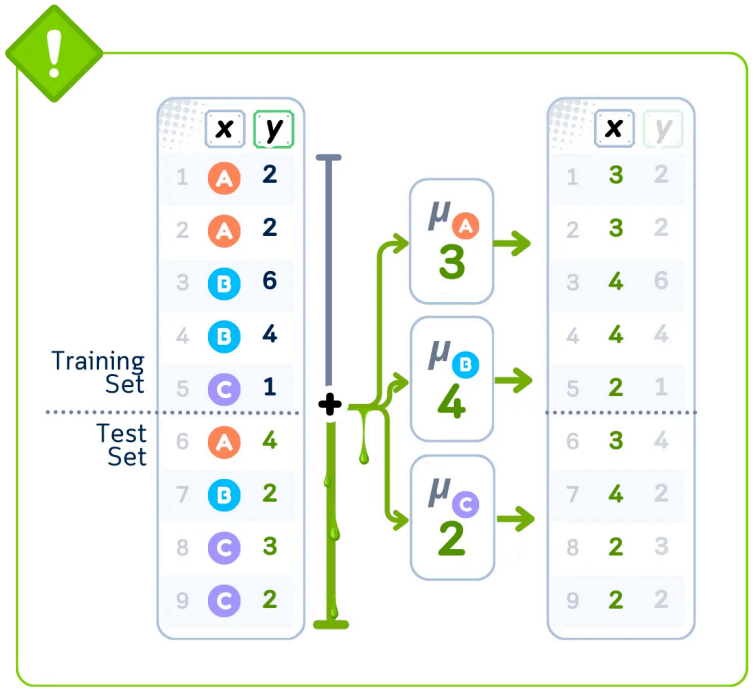


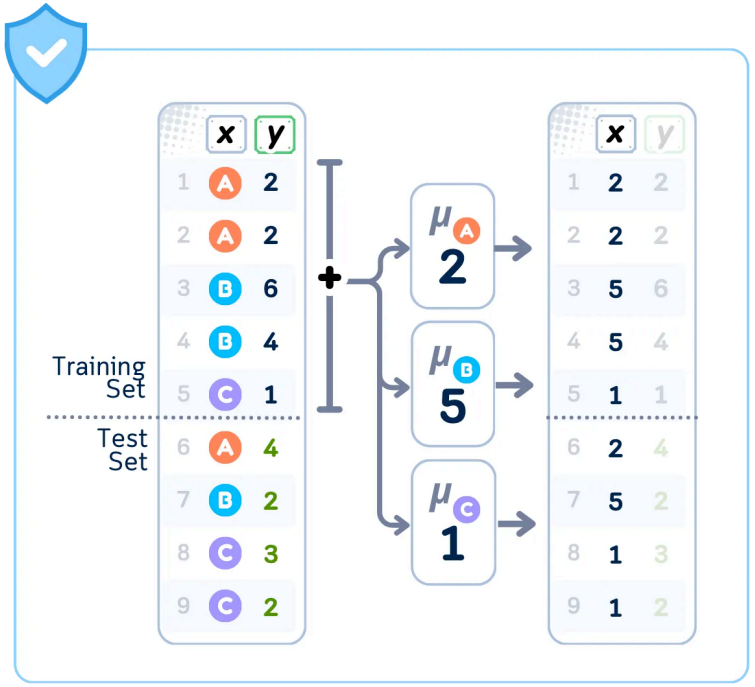
`TargetEncoder()`

🚨 EL PROBLEMA: càcul de categories amb el dataset al complet

❌ Què estem fent malament? Calcular substitucions de categories utilitzant valors ***target***

💥 La conseqüència: els atributs d'entrenament contenen informació de ***targets*** futurs



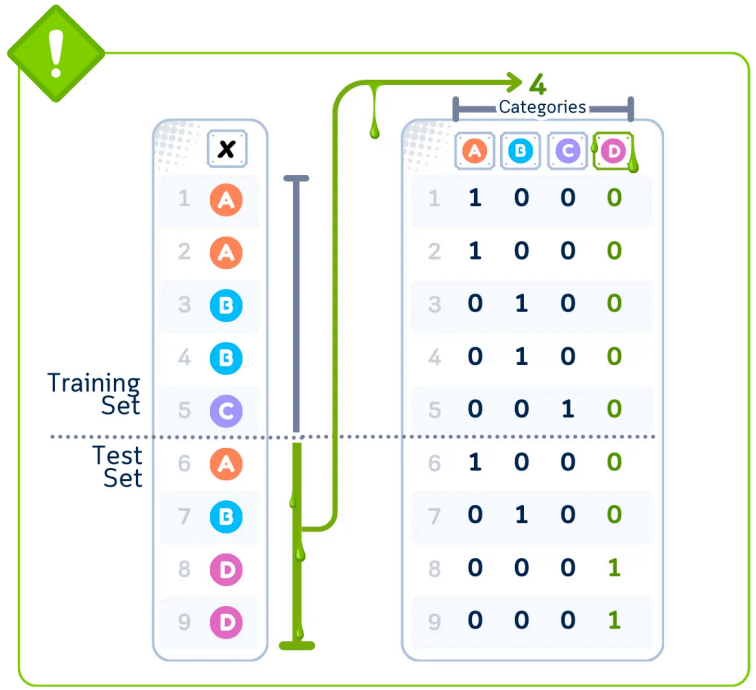


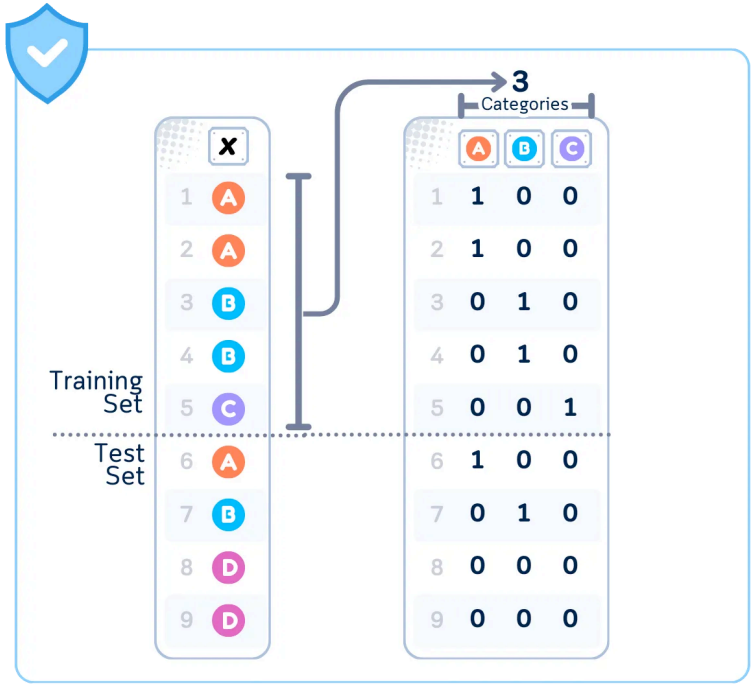
`OneHotEncoder()`

🚨 EL PROBLEMA: determinar categories a partir del conjunt del dataset complet

❌ Què estem fent malament? Crear columnes binàries basades en tots els valors únics

💥 La conseqüència: Seleccionem atributs influenciats pels patrons del conjunt de ***test***





Data scaling

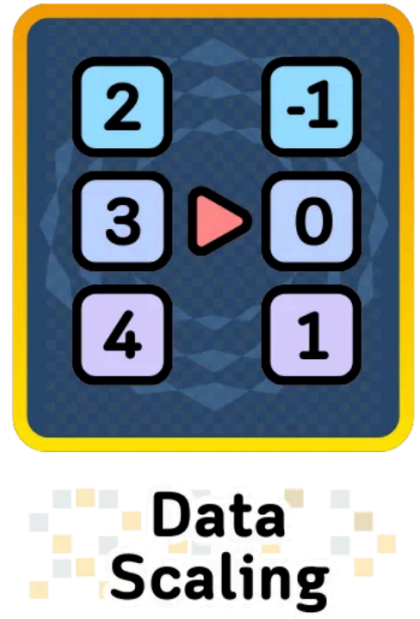
`StandardScaler()`

`MinMaxScaler()`

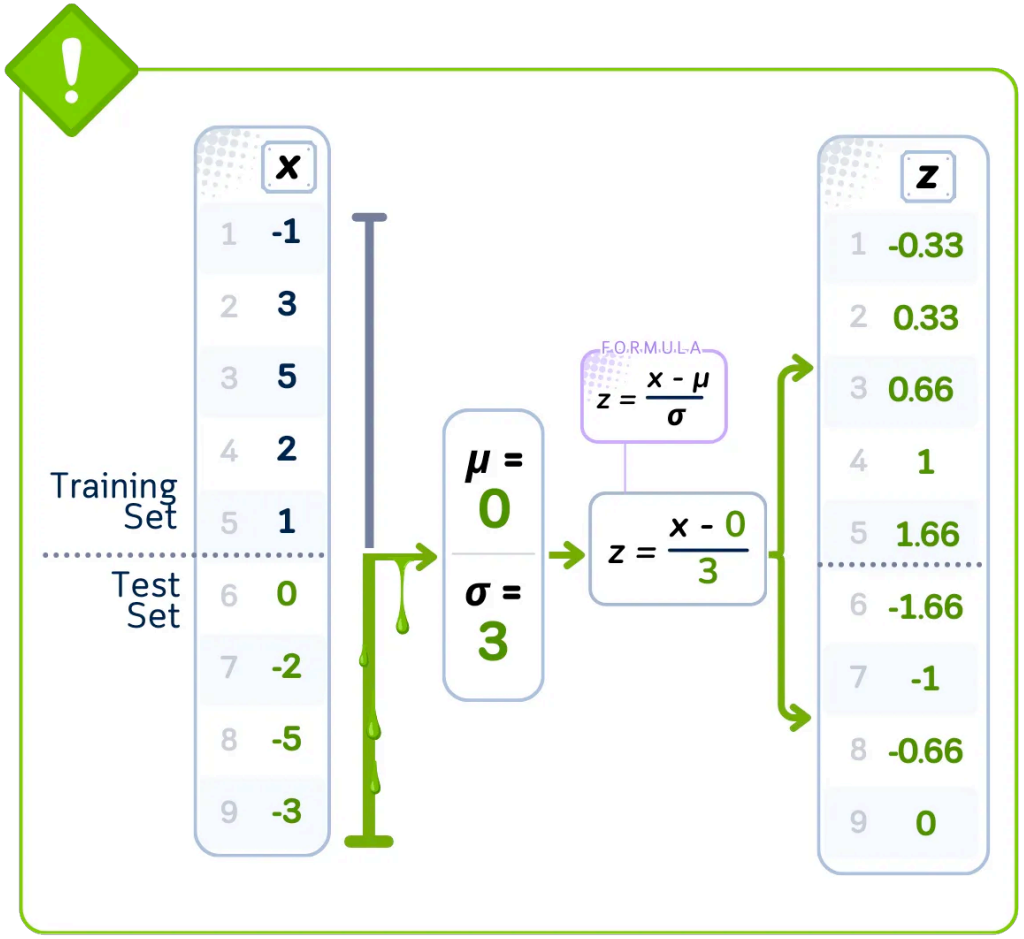
`RobustScaler()`

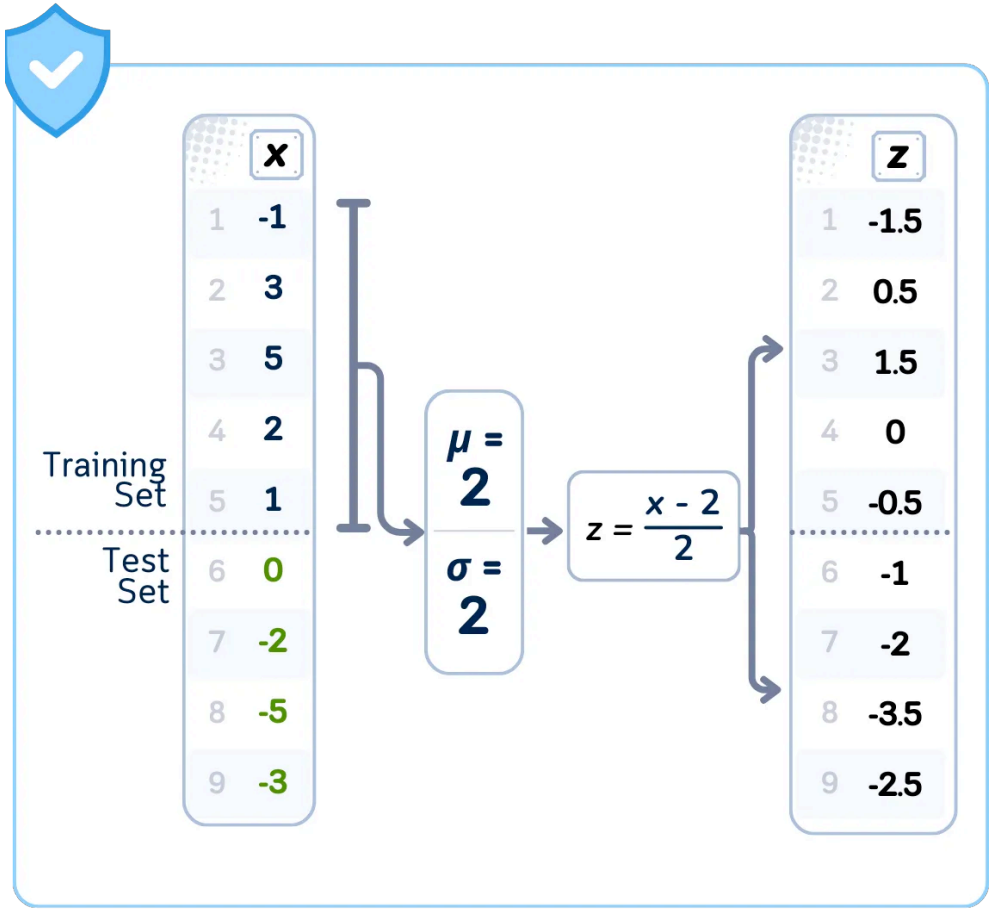
`FunctionTransformer(np.log1p)`

`PowerTransformer(method='box-cox')`

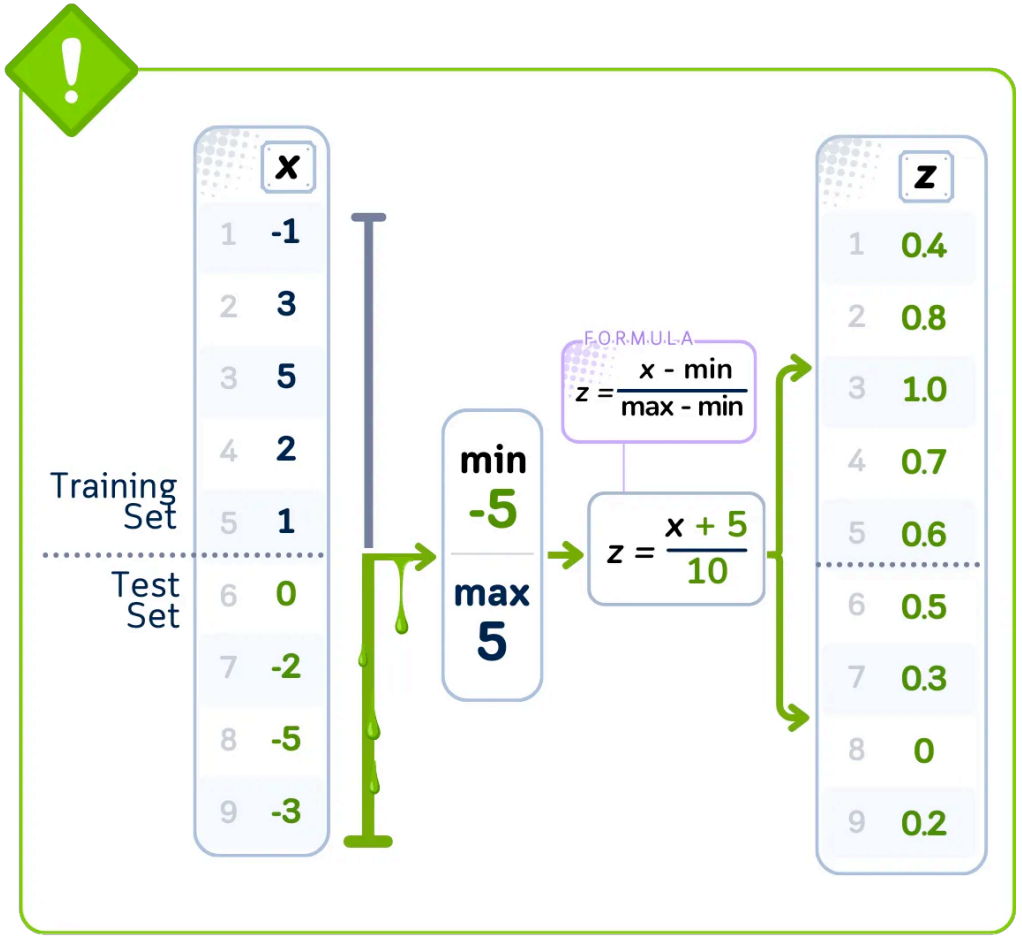


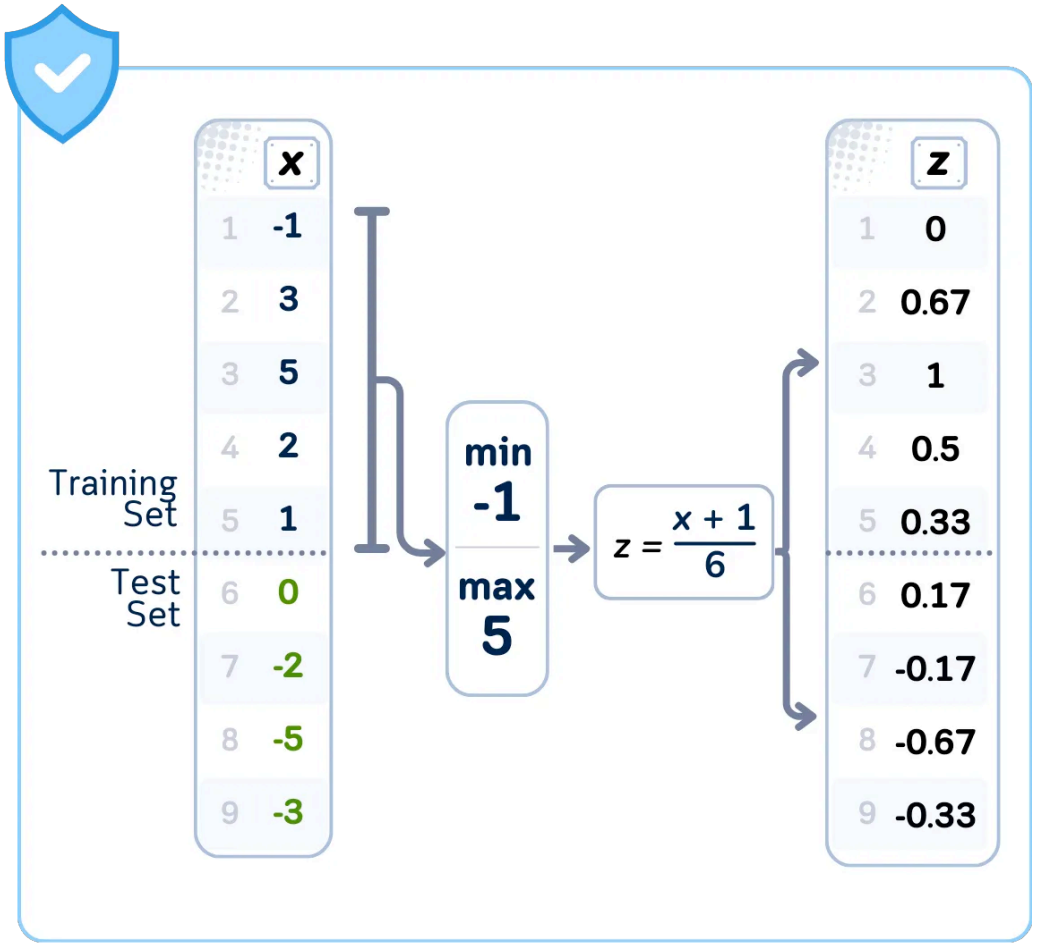
`StandardScaler()` (valors entre -1 i 1)





`MinMaxScaler()` (valors entre 0 i 1)

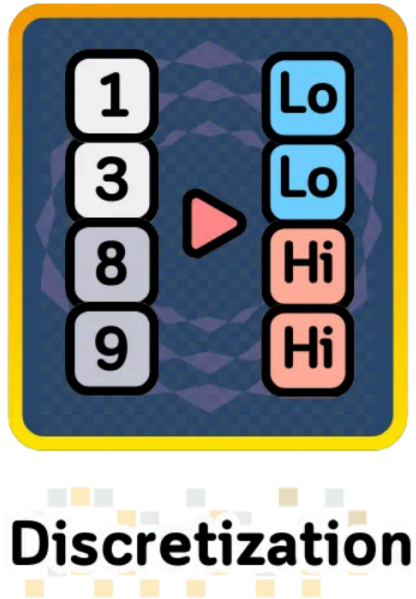




Discretization

A vegades és millor agrupar els números en categories en lloc d'utilitzar valors exactes.

Tot i que els valors d'agrupació poden ajudar els models a trobar patrons, la manera com decidim els límits ha d'utilitzar només dades d'entrenament per evitar filtracions.



Resampling

`RandomOverSampler()`

`SMOTE()`

`ADASYN()`

`RandomUnderSampler()`

`NearMiss(version=1)`

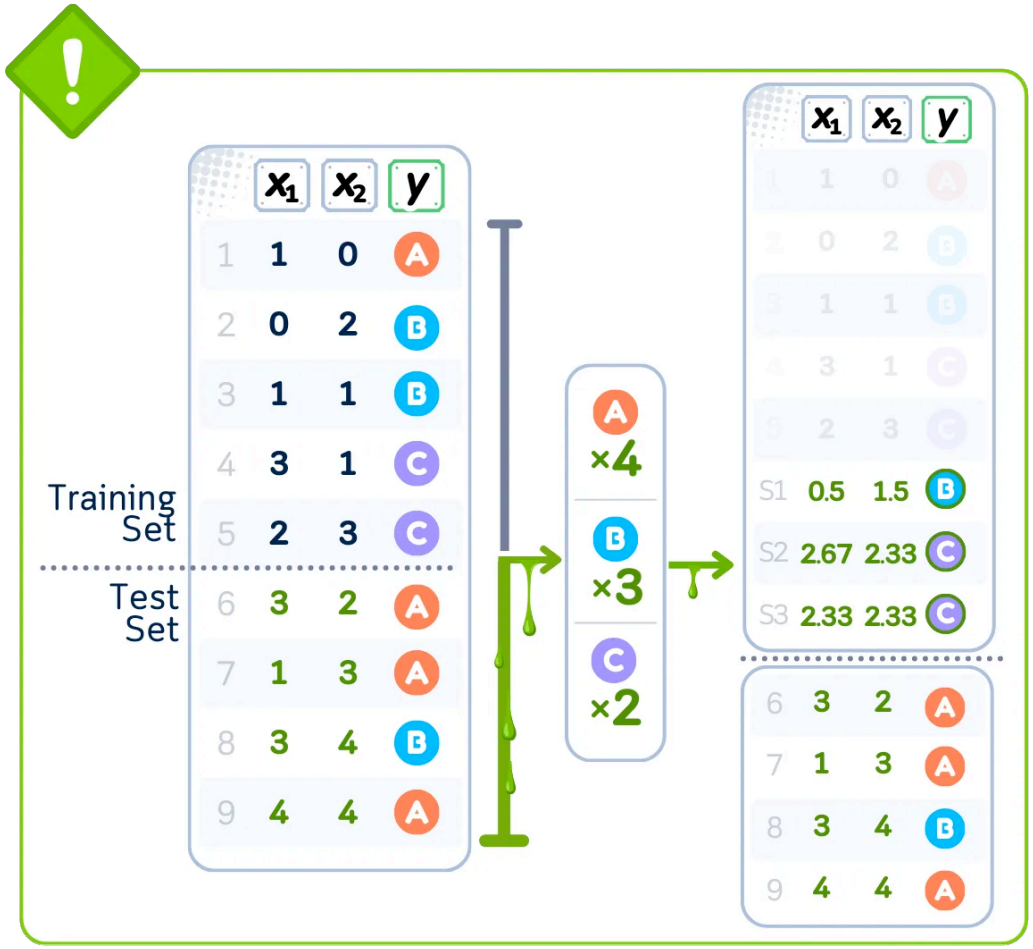
`NearMiss(version=2)`

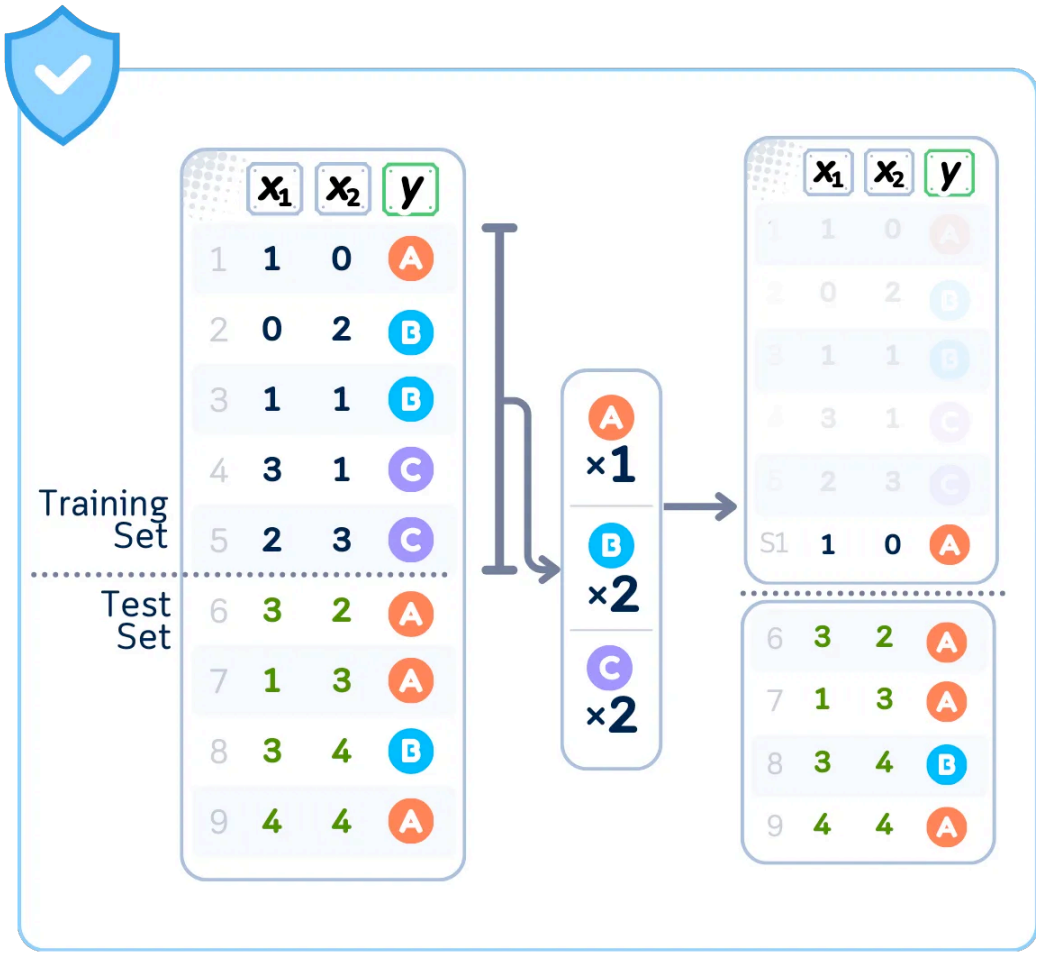
`TomekLinks()`

`EditedNearestNeighbours()`

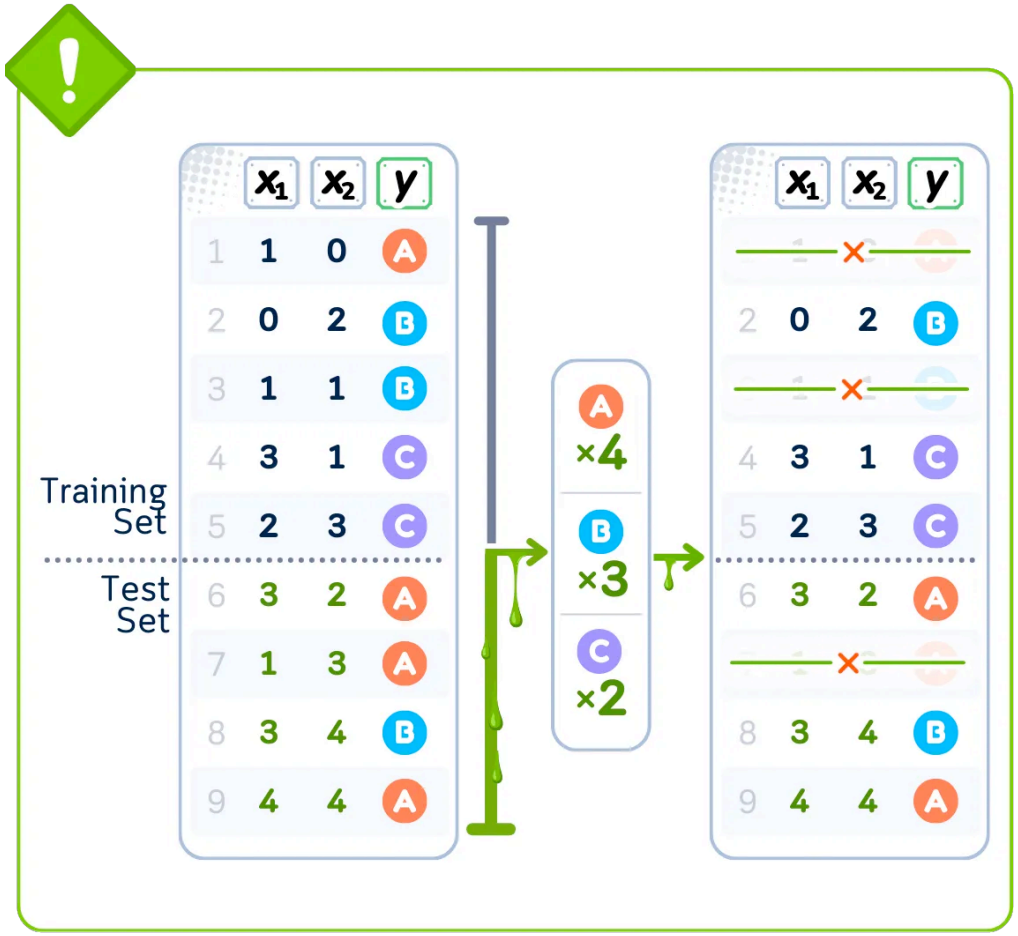


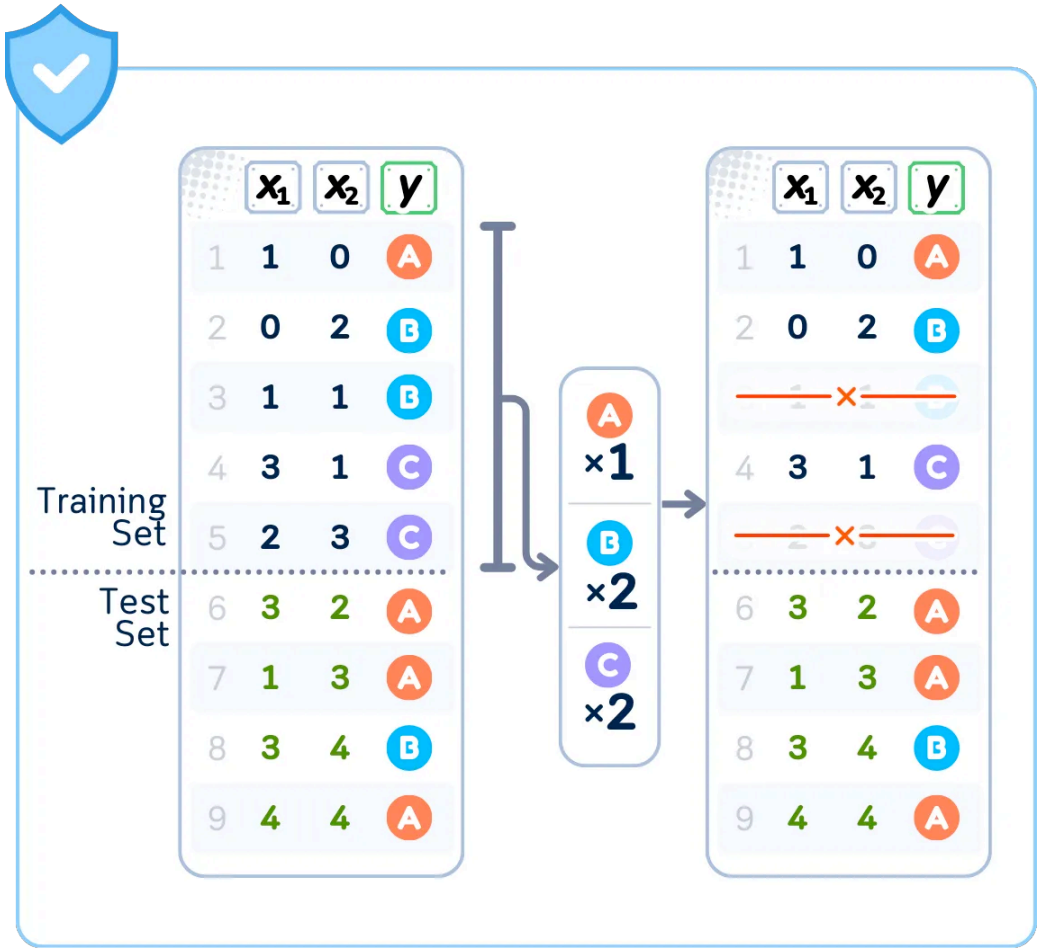
`SMOTE()` : crear nous exemples sintètics per a categories més petites amb interpolació





`TomekLinks()` : per seleccionar quins exemples s'han d'eliminar en funció de la seva similitud amb altres categories





Take-home message

Quan preprocesseu les dades, heu de mantenir les dades d'entrenament i de prova completament separades.

Cada vegada que utilitzeu informació de totes les dades per transformar valors, correu el risc de barrejar informació de *test* amb dades de *train*.

Solució

Transformeu sempre les vostres dades d'entrenament primer, deseu aquests càlculs i, després, apliqueu-los a les vostres dades de prova.

sklearn Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier())
])

pipeline.fit(X_train, y_train)  # Fit on training data
y_pred = pipeline.predict(X_test)  # Apply same preprocessing to test data
```

1. **Dividiu les dades primer, abans de qualsevol processament.**

2. **Totes les transformacions (preprocessament, SMOTE) es troben dins del *pipeline*.**

- ***Pipeline* garanteix:**

  - Paràmetres de preprocessament apresos només a partir de dades d'entrenament

  - SMOTE només s'aplica a les dades d'entrenament

  - Les dades de prova romanen completament invisibles fins a la predicció

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import load_iris

# Load the Iris dataset
data = load_iris()
X, y = data.data, data.target

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# Define a pipeline
pipeline = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),  # Handle missing values
    ('scaler', StandardScaler()),                 # Scale the features
    ('classifier', SVC())                         # Support Vector Classifier
])

# Define a parameter grid for GridSearchCV
param_grid = {
    'classifier__C': [0.1, 1, 10],                # Regularization parameter
    'classifier__kernel': ['linear', 'rbf'],      # Kernel type
    'classifier__gamma': [0.01, 0.1, 1]           # Kernel coefficient for 'rbf'
}
```

```
# Initialize GridSearchCV
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    verbose=1
)

# Fit the grid search to the training data
grid_search.fit(X_train, y_train)

# Print the best parameters and best score
print("Best Parameters:", grid_search.best_params_)
print("Best Cross-Validation Accuracy:", grid_search.best_score_)

# Evaluate on the test set
test_score = grid_search.score(X_test, y_test)
print("Test Set Accuracy:", test_score)
```